

PLAYWRIGHT vs SELENIUM

Two Engines, One Job

Why the locator and waiting model is the real story

The Real Difference Starts Below the API

Both frameworks click buttons and read text. How they talk to the browser is what changes everything else.

SELENIUM

- Test code calls the WebDriver client
- Sends a JSON command over HTTP (W3C wire protocol)
- A driver executable (chromedriver, geckodriver) receives it
- Driver translates and forwards to the browser
- Response makes the same round trip back

PLAYWRIGHT

- Test code calls the Playwright API directly
- Opens one persistent WebSocket connection
- Talks to the browser's own protocol (CDP for Chromium)
- No separate driver process in the middle
- Lower latency, fewer moving parts to break

The Pain You Already Know: Explicit Waits

Selenium doesn't know when an element is actually ready to use — you have to tell it, every time, in multiple steps.

`selenium-webdriver` (TypeScript)

```
import { Builder, By, until } from 'selenium-webdriver';

const driver = await new Builder().forBrowser('chrome').build();

// 1. Wait for the element to exist in the DOM
const button = await driver.wait(
  until.elementLocated(By.id('submit')),
  10000
);

// 2. Separately wait for it to be visible
await driver.wait(until.elementIsVisible(button), 10000);

// 3. Separately wait for it to be enabled
await driver.wait(until.elementIsEnabled(button), 10000);

// 4. Now, finally, click it
await button.click();
```

Playwright's Answer: Don't Make Me Ask

One line. Playwright checks visible, stable, enabled, and receives-events on its own — then acts.

Playwright (TypeScript)

```
await page.getByRole('button', { name: 'Submit' }).click();
```

What that one line does automatically before clicking:

- Waits for the element to be attached to the DOM
- Waits for it to be visible and stable (not mid-animation)
- Waits for it to be enabled and able to receive pointer events
- Retries the whole check internally until a timeout, no manual polling

A Locator Is a Recipe, Not a Result

This is the single biggest mental shift coming from Selenium.

SELENIUM: a resolved handle

`findElement()` runs once and hands you a reference bound to a specific DOM node, at that instant.

If the page re-renders before you use it...

StaleElementReferenceException

...thrown the moment you act on a reference the DOM left behind.

PLAYWRIGHT: a description

`locator()` doesn't find anything yet — it stores instructions for how to find it.

Every action re-resolves it fresh, live:

```
const btn = page.getByRole('button');  
await btn.click(); // re-finds, waits, acts  
await btn.click(); // does it again, fresh
```

Bonus Safety Net: Strict Mode

Selenium's `findElements()` silently hands back a list. Playwright treats ambiguity as a bug.

If a locator matches more than one element on `.click()` / `.fill()` / etc:

Playwright throws immediately — “strict mode violation: locator resolved to 3 elements.” Caught the first time that line runs, not silently clicking the wrong one in production.

Disambiguating on purpose:

```
await page.getByRole('listitem').first().click();
await page.getByRole('listitem').nth(2).click();
await page.getByRole('listitem').filter({ hasText: 'Pro plan' }).click();
```

Beyond Waiting: The Rest of the Picture

We'll go deep on locators today — here's the rest, at a glance, for later.

Topic	Selenium	Playwright
Browser protocol	WebDriver wire protocol + driver exe	Direct (CDP for Chromium), no driver exe
Parallelism	Needs Grid or 3rd-party runner config	Built into the test runner by default
Visual diffing	Not built in — bolt on Percy/Applitools	Native: <code>expect(page).toHaveScreenshot()</code>
Debugging a failure	Re-run locally, add prints, hope	Trace Viewer: replay the exact CI run
Native mobile apps	Pairs with Appium	Not supported — web/Electron only
Language bindings	JS, Python, Java, C#, Ruby, more	JS/TS, Python, Java, .NET

The Takeaway Before We Touch Code

It's not that Selenium is wrong — it's that the waiting and locating model was built for an older browser-automation protocol. Playwright's lazy locator + auto-wait removes an entire category of flaky-test bugs you've been manually working around for years.

Next: installing the extension, the test runner, and getting your hands on it.